GitHub Actions for CI/CD

Complete Implementation Guide - Lab 5 (AZ-400)

Prepared by: Zareena

What is GitHub Actions?

GitHub Actions Overview:

GitHub Actions is a powerful CI/CD platform that allows you to automate your software development workflows directly within your GitHub repository. It enables you to build, test, and deploy your code automatically based on events like commits, pull requests, or scheduled triggers.

Key Features

- **Event-Driven Automation:** Trigger workflows based on GitHub events (push, pull request, issues, etc.)
- **Native Integration:** Seamlessly integrated with GitHub repositories and services
- **Marketplace:** Access thousands of pre-built actions from the community
- **Matrix Builds:** Test across multiple operating systems and versions simultaneously
- **Secrets Management:** Securely store and use sensitive information like API keys and credentials
- **Self-Hosted Runners:** Run workflows on your own infrastructure when needed

Core Components

1. **Workflows:** Automated processes defined in YAML files that run in your repository
2. **Events:** Specific activities that trigger workflow runs (push, pull_request, schedule, etc.)
3. **Jobs:** Sets of steps that execute on the same runner
4. **Steps:** Individual tasks that run commands or actions
5. **Actions:** Reusable units of code that perform specific tasks
6. **Runners:** Servers that execute your workflows (GitHub-hosted or self-hosted)

CI/CD with GitHub Actions

What is CI/CD?

Continuous Integration (CI):

The practice of automatically building and testing code changes whenever developers commit to the repository. This helps identify bugs early and ensures code quality.

Continuous Deployment/Delivery (CD):

The automated process of deploying tested code changes to production or staging environments. This reduces manual intervention and speeds up the release cycle.

CI/CD Pipeline Stages

1. **Source:** Code is committed and pushed to GitHub repository
2. **Build:** Application is compiled and dependencies are installed
3. **Test:** Automated tests run to verify code quality and functionality
4. **Deploy:** Application is deployed to target environment (staging/production)
5. **Monitor:** Application performance and health are monitored continuously

Benefits of Using GitHub Actions for CI/CD

⚡ Speed

Faster development cycles with automated workflows

Reliability

Consistent and repeatable deployment processes

Security

Built-in secrets management and secure authentication

Cost-Effective

Free for public repositories, affordable for private ones

Flexibility

Customize workflows to match your exact needs

Integration

Native GitHub integration and extensive marketplace

GitHub Actions Workflow Structure

Basic Workflow YAML Structure

```
name: CI/CD Pipeline on: push: branches: [ main ] pull_request:
branches: [ main ] workflow_dispatch: env: AZURE_WEBAPP_NAME: my-app
RESOURCE_GROUP: rg-myapp jobs: build: runs-on: ubuntu-latest steps: -
uses: actions/checkout@v3 - name: Setup .NET uses: actions/setup-
dotnet@v3 - name: Build run: dotnet build - name: Test run: dotnet
test deploy: needs: build runs-on: ubuntu-latest environment:
Production steps: - name: Azure Login uses: azure/login@v1 with:
creds: ${{ secrets.AZURE_CREDENTIALS }} - name: Deploy to Azure run:
az webapp deploy
```

Workflow Syntax Explained

- `name:` - Human-readable name for your workflow
- `on:` - Events that trigger the workflow
- `env:` - Environment variables available to all jobs
- `jobs:` - Collection of jobs that run in the workflow
- `runs-on:` - Type of runner (ubuntu-latest, windows-latest, etc.)
- `steps:` - Sequence of tasks within a job
- `uses:` - Specifies an action to run
- `with:` - Input parameters for actions

- `secrets:` - Access to encrypted repository secrets

 Best Practice: Store workflow files in the `.github/workflows/` directory of your repository. You can have multiple workflow files for different purposes (CI, CD, testing, etc.).

GitHub Actions + Azure Integration

Why Integrate with Azure?

GitHub Actions provides excellent integration with Azure services, allowing you to:

- Deploy web applications to Azure App Service
- Manage infrastructure using Azure Resource Manager templates
- Interact with Azure services using Azure CLI or PowerShell
- Implement secure authentication using Azure Service Principals
- Automate container deployments to Azure Container Registry

Common Azure Actions

1. `azure/login@v1` - Authenticate with Azure using service principal
2. `azure/webapps-deploy@v2` - Deploy to Azure Web Apps
3. `azure/arm-deploy@v1` - Deploy ARM templates for infrastructure
4. `azure/docker-login@v1` - Login to Azure Container Registry

Authentication Methods:

- **Service Principal:**

  Traditional approach using client ID and secret

- **OpenID Connect (OIDC):**

  Modern secretless authentication (recommended)

Lab Overview

What You'll Learn:

In this lab, you'll learn how to implement a GitHub Action workflow that deploys an Azure web app using CI/CD principles.

Learning Objectives

- Implement a GitHub Action workflow for CI/CD
- Explain the basic characteristics of GitHub Action workflows
- Deploy Azure resources using GitHub Actions
- Configure manual approval processes for deployments

Estimated Time:

40 minutes

Lab Requirements

Browser

☁ Azure Subscription

Microsoft Edge or Azure DevOps supported browser

Active Azure subscription with appropriate permissions

GitHub Account

Valid GitHub account for repository management

Permissions

Contributor or Owner role in Azure subscription

1 Exercise 1: Import eShopOnWeb Repository

Repository Structure

The eShopOnWeb repository is organized as follows:

- `.ado` - Azure DevOps YAML pipelines
- `.devcontainer` - Container setup for VS Code/GitHub Codespaces
- `infra` - Bicep & ARM infrastructure as code templates
- `.github` - YAML GitHub workflow definitions
- `src` - .NET 8 website source code

Task 1: Create GitHub Repository

1. Navigate to `https://github.com` and sign in
2. Click New to create a new repository
3. Import the existing eShopOnWeb repository code to your private repo

2 Create Azure Service Principal

 Note: In this Cloudslice lab, a Resource Group named `rg-eshoponweb` has already been created for you.

Task Steps

1. Open the Azure Portal at `https://portal.azure.com`

2. In the Azure Portal, open the Cloud Shell (next to the search bar)

⚠ **Storage Setup:** If Azure Portal asks you to create storage, choose "No storage account required", select your subscription, and click Apply.

3. Ensure the terminal is running in Bash mode

4. Execute the following command (replace SUBSCRIPTION-ID and RESOURCE-GROUP):

```
az ad sp create-for-rbac --name GH-Action-eshoponweb55871541 \ --role contributor \ --scopes /subscriptions/SUBSCRIPTION-ID/resourceGroups/RESOURCE-GROUP \ --sdk-auth
```

Important: This command must be typed or pasted as a single line!

5. Copy the JSON output - you'll use it as a GitHub secret

JSON Output Structure:

```
{ "clientId": "<GUID>", "clientSecret": "<GUID>",
"subscriptionId": "<GUID>", "tenantId": "<GUID>" }
```

6. Register the Azure App Service resource provider:

```
az provider register --namespace Microsoft.Web
```

7. In your GitHub repository, go to Settings → Secrets and variables → Actions

8. Click New repository secret

9. Create the secret:

- **Name:** `AZURE_CREDENTIALS`
- **Secret:** Paste the JSON object you copied earlier

10. Click Add secret

3 Modify and Execute GitHub Workflow

Task 2: Configure the Workflow

1. Navigate to your eShopOnWeb GitHub repository

2. Go to Code and open: `eShopOnWeb/.github/workflows/eshoponweb-cicd.yml`

3. Uncomment the `on` section (delete "#")

Workflow Triggers: The workflow triggers with every push to the main branch and also offers manual triggering (workflow_dispatch).

4. In the `env` section, make these changes:

Required Changes:

- **RESOURCE-GROUP:**

  Replace NAME with `rg-eshoponweb`

- **LOCATION:**

  Use `westus2`

- **SUBSCRIPTION-ID:**

  Replace YOUR-SUBS-ID with `a50cc65a-2489-414e-8382-17d214c9a6f9`

5. Commit the changes (this will trigger the workflow automatically)

Task 3: Review Workflow Execution

1. Go to the Actions tab in your repository

2. Click on the workflow execution to see the details

3. Wait for the workflow to complete

Workflow Summary:

You'll see two workflow jobs with their status and artifacts. Click on each job to review logs and details.

4 Add Manual Approval (Optional)

Task 4: Configure GitHub Environments

1. Open the workflow file: `eShopOnWeb/.github/workflows/eshoponweb-cicd.yml`

2. In the `deploy` job section, find the reference to the Development environment

GitHub Environments: GitHub uses environments to add protection rules and secrets for your deployment targets.

3. Go to Settings → Environments → New environment

4. Name it `Development` and click Configure Environment

5. Check the option Required Reviewers and add your GitHub account as a reviewer

6. Click Save protection rules

7. Manually run the workflow or trigger it by pushing to main

8. Wait for the `buildandtest` job to finish - you'll see a review request

9. Click Review deployments, check Development, and click Approve and deploy

10. The workflow will continue with the deploy job execution

Cleanup

⚠ **Important:** Remember to delete the resources created in the Azure portal to avoid unnecessary charges.

Lab Review

What You've Accomplished:

In this lab, you successfully implemented a GitHub Action workflow that deploys an Azure Web App using CI/CD principles, including:

- Setting up GitHub repository with eShopOnWeb code
- Creating and configuring Azure Service Principal
- Modifying and executing GitHub workflows

- Implementing manual approval processes for deployments

Congratulations!

You have successfully completed the GitHub Actions for CI/CD lab.

AZ-400 DevOps Engineer Expert Certification Path